

Lesson 12 Structured Note: Extended Parenthesis and Bracket Matching

0. Current Position in the Course

You are now in **Lesson 12** of the stack curriculum.

Before Lesson 12, you already learned:

Lesson	Main idea
Lesson 1	A stack follows LIFO: Last In, First Out .
Lesson 2	An array stack uses <code>size</code> , <code>top</code> , and an array pointer.
Lesson 3	Array stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 4	A complete C array-stack program combines all array-stack functions.
Lesson 5	A linked-list stack uses nodes and a <code>top</code> pointer.
Lesson 6	Linked-list stack operations include <code>push</code> , <code>pop</code> , <code>peek</code> , <code>stackTop</code> , <code>isEmpty</code> , and <code>isFull</code> .
Lesson 7	A complete C linked-list stack uses <code>struct Node</code> , a global <code>top</code> pointer, <code>push</code> , <code>pop</code> , <code>display</code> , and <code>main</code> .
Lesson 8	A C++ class can be used to implement a linked-list stack more safely.
Lesson 9	Parenthesis matching was introduced as a stack application.
Lesson 10	Parenthesis matching was implemented using an array character stack.
Lesson 11	Parenthesis matching was implemented using a linked-list character stack.
Lesson 12	Parenthesis matching is extended to multiple bracket types: <code>()</code> , <code>[]</code> , and <code>{}</code> .

Lesson 12 is still part of:

Phase 3: Parenthesis and Bracket Matching

In Lesson 9, 10, and 11, we mainly checked only round parentheses:

```
( )
```

In Lesson 12, we now check three types of brackets:

```
( )  
[ ]  
{ }
```

The stack rule does not change.

The rule is still:

```
Last In, First Out
```

The main improvement is that the algorithm must now check both:

1. correct order
2. correct bracket type

1. Lesson 12 Goal

By the end of this lesson, you should understand:

- What extended bracket matching means.
- Why checking only `()` is not enough.
- How to check `()`, `[]`, and `{}` together.
- Why stacks are useful for nested brackets.
- Why opening brackets are pushed onto the stack.
- Why closing brackets cause a pop operation.
- Why the popped opening bracket must match the current closing bracket.
- Why bracket count alone is not enough.
- How to detect an extra opening bracket.
- How to detect an extra closing bracket.
- How to detect a bracket-type mismatch.
- How the ASCII matching trick works.
- Why the ASCII trick must be used carefully.
- How to write the algorithm in plain English.
- How to dry-run balanced and unbalanced examples.
- How to write a beginner C++ implementation using `stack<char>`.
- How to write the matching logic with and without the ASCII trick.
- The time and space complexity of the algorithm.

- Common beginner mistakes.
-

2. Quick Review: What Is a Stack?

A stack is a data structure that follows this rule:

```
LIFO = Last In, First Out
```

This means:

```
The last item inserted is the first item removed.
```

Example:

```
push(10)
push(20)
push(30)
```

Logical stack view:

```
Top -> 30
      20
      10
```

If we call:

```
pop()
```

The removed value is:

```
30
```

Why?

Because `30` was inserted last, so it must be removed first.

This same idea helps us with nested brackets.

3. What Problem Are We Solving in Lesson 12?

Before Lesson 12, we checked only round parentheses:

()

Example:

((A+B) * (C-D))

Now we want to check all bracket types:

()
[]
{ }

Example:

{([A+B]*[C-D])}

This expression uses:

{
(
[
]
[
]
)
}

We need to check whether every opening bracket has the correct closing bracket.

4. What Does Balanced Mean?

An expression is balanced when all of these are true:

1. Every opening bracket has a matching closing bracket.

2. Every closing bracket has a previous unmatched opening bracket.
3. Brackets close in the correct order.
4. The opening and closing bracket types match.
5. No opening bracket is left over at the end.
6. No closing bracket appears too early.

Balanced example:

```
{([A+B]*[C-D])}
```

This is balanced because:

- { is closed by }.
- (is closed by).
- [is closed by].
- The brackets close in the correct order.

Unbalanced example:

```
{([A+B]*[C-D])}
```

This is not balanced because:

[was opened, but) tries to close it.

A square bracket [must be closed by], not by).

5. The Three Bracket Types

Lesson 12 checks these three bracket types:

Opening bracket	Closing bracket	Name
(	)	Round parentheses
[	]	Square brackets
{	}	Curly braces

These are the only valid matching pairs:

```
( matches )  
[ matches ]  
{ matches }
```

These are invalid pairs:

```
( does not match ]  
( does not match }  
[ does not match )  
[ does not match }  
{ does not match }  
{ does not match ]
```

6. Why a Stack Is Useful for Bracket Matching

Nested brackets follow the stack rule.

Look at this expression:

```
{([A+B])}
```

The brackets open in this order:

```
{  
(  
[
```

The most recent opening bracket is:

```
[
```

So it must close first:

```
]
```

Then `(` must close:

)

Then { must close:

}

So the closing order is the reverse of the opening order.

Opening order:

{ ([

Closing order:

]) }

This is exactly:

Last In, First Out

That is why we use a stack.

7. Main Stack Rule for Lesson 12

We scan the expression from left to right.

If we see an opening bracket:

(or [or {

we push it onto the stack.

If we see a closing bracket:

) or] or }

we pop the top opening bracket from the stack and check whether it matches.

Simple rule:

```
Opening bracket -> push  
Closing bracket -> pop and check type  
Other character -> ignore
```

8. Characters We Care About

In Lesson 12, we care about only these characters:

```
( ) [ ] { }
```

Everything else is ignored.

Ignored characters include:

```
A, B, C, D  
numbers  
+  
-  
*  
/  
spaces
```

Example:

```
{([A+B]*[C-D])}
```

The algorithm only cares about:

```
{ ( [ ] [ ] ) }
```

It ignores:

```
A + B * C - D
```

9. Why Count Alone Is Not Enough

A beginner may think:

If the number of opening brackets equals the number of closing brackets, the expression is balanced.

But this is wrong.

Example:

([])

Count the brackets:

Opening brackets: ([
Closing brackets:)]

There are 2 opening brackets and 2 closing brackets.

The count looks correct.

But the expression is still not balanced.

Why?

Let us dry-run it:

Character	Action	Stack after action
(	push	(
[	push	([
)	pop [	mismatch

At the third character, we see:

)

But the top of the stack is:

```
[
```

This is wrong because:

```
[ must close with ]
```

It cannot close with:

```
)
```

Important idea:

```
Balanced brackets are not only about count.  
They are about correct type and correct order.
```

10. Correct Order Matters

This is balanced:

```
([])
```

Why?

Opening order:

```
(  
[
```

Closing order:

```
]
)
```

The most recent opening bracket `[` closes first.

This is not balanced:

```
( [ ] )
```

Why?

Opening order:

```
(  
[
```

But the first closing bracket is:

```
)
```

This tries to close `(` before closing `[`, but `[` is on top of the stack.

So the order is wrong.

11. Types of Errors in Lesson 12

There are three main error types.

11.1 Extra Opening Bracket

An extra opening bracket means an opening bracket was never closed.

Example:

```
{ ([A+B])
```

Here, `{` is opened but never closed.

At the end, the stack is not empty.

Result:

```
Not Balanced
```

This error is detected at the final stack check.

11.2 Extra Closing Bracket

An extra closing bracket means a closing bracket appears without a previous unmatched opening bracket.

Example:

```
(A+B)]
```

At some point, the algorithm sees:

```
]
```

But the stack is already empty.

There is no opening bracket available to match it.

Result:

```
Not Balanced
```

This error is detected immediately while scanning.

11.3 Type Mismatch

A type mismatch means the current closing bracket does not match the most recent opening bracket.

Example:

```
{([A+B)*[C-D])}
```

At the mismatch point:

```
[ was opened, but ) appeared.
```

This is wrong because:

```
[ matches ]
```

not:

```
)
```

Result:

```
Not Balanced
```

This error is detected when we pop and compare.

12. Matching Pairs

When we pop an opening bracket, we compare it with the current closing bracket.

Valid examples:

```
popped '(' and current ')' -> valid  
popped '[' and current ']' -> valid  
popped '{' and current '}' -> valid
```

Invalid examples:

```
popped '[' and current ')' -> invalid  
popped '(' and current ']' -> invalid  
popped '{' and current ')' -> invalid
```

The popped bracket must be the exact matching opening bracket for the current closing bracket.

13. The ASCII Matching Trick

Lesson 12 introduces a shortcut using ASCII values.

In ASCII:

```
')' - '(' = 1  
']' - '[' = 2  
'}' - '{' = 2
```

So for valid bracket pairs, the difference is either:

1 or 2

Example code:

```
int diff = closing - opening;
```

Then we check:

```
return diff == 1 || diff == 2;
```

If the difference is 1 or 2, the pair is valid.

14. ASCII Trick Examples

Example 1

```
opening = '('  
closing = ')'
```

Difference:

```
')' - '(' = 1
```

Since the difference is 1, this is valid.

Example 2

```
opening = '['  
closing = ']'
```

Difference:

```
']' - '[' = 2
```

Since the difference is `2`, this is valid.

Example 3

```
opening = '{'  
closing = '}'
```

Difference:

```
'}' - '{' = 2
```

Since the difference is `2`, this is valid.

Example 4

```
opening = '['  
closing = ')'
```

This should be invalid.

A square bracket cannot be closed by a round parenthesis.

15. Important Warning About the ASCII Trick

The ASCII trick is useful, but do not use it blindly.

You must still first check whether a character is:

1. an opening bracket
2. a closing bracket
3. something else

Why?

Because the ASCII trick only makes sense when comparing one opening bracket with one closing bracket.

So the algorithm should still use helper functions like:

```
isOpening(ch)
isClosing(ch)
isMatching(opening, closing)
```

The ASCII trick helps inside `isMatching()`, but it does not replace the whole algorithm.

16. Algorithm in Simple Words

To check whether an expression is balanced:

Create an empty stack.

Scan the expression from left to right.

For each character:

If the character is '(', '[', or '{':
Push it onto the stack.

Else if the character is ')', ']', or '}':
If the stack is empty:
Return false.

Otherwise:
Pop the top opening bracket.
Check whether the popped opening bracket matches the current closing bracket.

If they do not match:
Return false.

Else:
Ignore the character.

After scanning the full expression:

If the stack is empty:
Return true.

```
Otherwise:  
    Return false.
```

17. Algorithm as Pseudocode

```
isBalanced(expression):  
    create an empty stack  
  
    for each character ch in expression:  
  
        if ch is an opening bracket:  
            push ch onto the stack  
  
        else if ch is a closing bracket:  
            if stack is empty:  
                return false  
  
            opening = stack top  
            pop stack  
  
            if opening does not match ch:  
                return false  
  
        else:  
            ignore ch  
  
    if stack is empty:  
        return true  
    else:  
        return false
```

18. Why We Check Empty Before Popping

When we see a closing bracket, we need a previous opening bracket to match it.

Example:

```
A+B)
```

The algorithm sees:

```
)
```

But the stack is empty.

That means there is no previous opening bracket.

So the expression is not balanced.

This is similar to stack underflow:

```
Trying to pop from an empty stack.
```

So before popping, we must check:

```
Is the stack empty?
```

If yes, return false immediately.

19. Why the Final Stack Check Is Necessary

Suppose the expression is:

```
{ ([A+B])
```

During scanning, there may be no extra closing bracket.

So the algorithm does not fail early.

But at the end, the stack still contains:

```
{
```

That means one opening bracket was never closed.

So the final check must be:

```
return stack is empty
```

If the stack is empty, the expression is balanced.

If the stack is not empty, the expression is not balanced.

20. Balanced Dry Run

Expression:

```
{([A+B]*[C-D])}
```

Only brackets matter:

```
{ ( [ ] [ ] ) }
```

Initial stack:

```
empty
```

Trace:

Step	Character	Action	Stack after action
1	{	push {	{
2	(	push (	{ (
3	[	push [	{ ([
4	A+B	ignore	{ ([
5	]	pop [and match	{ (
6	*	ignore	{ (
7	[	push [	{ ([
8	C-D	ignore	{ ([
9	]	pop [and match	{ (
10	)	pop (and match	{
11	}	pop { and match	empty

Final stack:

empty

Result:

Balanced

Why?

Every opening bracket got the correct closing bracket in the correct order.

21. Mismatch Dry Run

Expression:

{([A+B)*[C-D])}

Only important bracket part:

{ ([) }

Initial stack:

empty

Trace:

Step	Character	Action	Stack after action
1	{	push {	{
2	(	push (	{ (
3	[	push [	{ ([
4	A+B	ignore	{ ([
5	)	pop [	mismatch

At step 5, the current closing bracket is:

)

But the most recent opening bracket is:

[

This is wrong.

A [must close with:

]

not with:

)

Result:

Not Balanced

22. Extra Opening Bracket Dry Run

Expression:

{([A+B])

Initial stack:

empty

Trace:

Step	Character	Action	Stack after action
1	{	push {	{

Step	Character	Action	Stack after action
2	(	push (	{ (
3	[	push [	{ ([
4	A+B	ignore	{ ([
5	]	pop [and match	{ (
6	)	pop (and match	{

At the end:

Stack still contains {

So the expression is:

Not Balanced

Reason:

The opening { was never closed.

23. Extra Closing Bracket Dry Run

Expression:

(A+B)]

Initial stack:

empty

Trace:

Step	Character	Action	Stack after action
1	(	push (	(

Step	Character	Action	Stack after action
2	A+B	ignore	(
3	)	pop (and match	empty
4	]	stack is empty, cannot pop	error

At step 4, the algorithm sees:

]

But the stack is empty.

So there is no opening bracket available to match it.

Result:

Not Balanced

Reason:

There is an extra closing bracket.

24. Beginner C++ Program Using `stack<char>`

This version uses the built-in C++ stack.

The goal is to focus on the Lesson 12 logic without writing your own stack implementation again.

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isOpening(char ch) {
    return ch == '(' || ch == '[' || ch == '{';
}

bool isClosing(char ch) {
    return ch == ')' || ch == ']' || ch == '}';
}
```

```

}

bool isMatching(char opening, char closing) {
    int diff = closing - opening;
    return diff == 1 || diff == 2;
}

bool isBalanced(string exp) {
    stack<char> st;

    for (char ch : exp) {
        if (isOpening(ch)) {
            st.push(ch);
        }
        else if (isClosing(ch)) {
            if (st.empty()) {
                return false;
            }

            char opening = st.top();
            st.pop();

            if (!isMatching(opening, ch)) {
                return false;
            }
        }
    }

    return st.empty();
}

int main() {
    string exp = "{([A+B]*[C-D])}";

    if (isBalanced(exp)) {
        cout << "Balanced" << endl;
    } else {
        cout << "Not Balanced" << endl;
    }

    return 0;
}

```

Expected output:

Balanced

25. Code Explanation

25.1 Header Files

```
#include <iostream>
#include <stack>
#include <string>
```

Meaning:

Header	Purpose
<code><iostream></code>	Used for <code>cout</code> .
<code><stack></code>	Gives access to the built-in C++ <code>stack</code> .
<code><string></code>	Allows us to use the <code>string</code> type.

25.2 `isOpening()` Function

```
bool isOpening(char ch) {
    return ch == '(' || ch == '[' || ch == '{';
}
```

This function checks whether a character is an opening bracket.

It returns `true` if `ch` is:

```
(
[
{
```

Otherwise, it returns `false` .

25.3 isClosing() Function

```
bool isClosing(char ch) {  
    return ch == ')' || ch == ']' || ch == '}';  
}
```

This function checks whether a character is a closing bracket.

It returns `true` if `ch` is:

```
)  
]  
}
```

Otherwise, it returns `false`.

25.4 isMatching() Function with ASCII Trick

```
bool isMatching(char opening, char closing) {  
    int diff = closing - opening;  
    return diff == 1 || diff == 2;  
}
```

This function checks whether the opening bracket and closing bracket match.

It uses the ASCII trick:

```
')' - '(' = 1  
']' - '[' = 2  
'}' - '{' = 2
```

So if the difference is `1` or `2`, the pair is considered valid.

25.5 isBalanced() Function

```
bool isBalanced(string exp) {  
    stack<char> st;
```

```

    for (char ch : exp) {
        if (isOpening(ch)) {
            st.push(ch);
        }
        else if (isClosing(ch)) {
            if (st.empty()) {
                return false;
            }

            char opening = st.top();
            st.pop();

            if (!isMatching(opening, ch)) {
                return false;
            }
        }
    }

    return st.empty();
}

```

This is the main function.

It scans the expression one character at a time.

If it sees an opening bracket, it pushes it.

If it sees a closing bracket, it checks the stack.

If the stack is empty, it returns `false`.

If the stack is not empty, it pops the top opening bracket and checks whether it matches the closing bracket.

At the end, it returns:

```
st.empty()
```

This means:

```

If the stack is empty, the expression is balanced.
If the stack is not empty, the expression is not balanced.

```

26. Matching Function Without ASCII Trick

For beginners, this version may be easier to understand:

```
bool isMatching(char opening, char closing) {  
    return (opening == '(' && closing == ')') ||  
           (opening == '[' && closing == ']') ||  
           (opening == '{' && closing == '}');  
}
```

This version directly lists all valid pairs.

It means:

If opening is '(' and closing is ')', return true.
If opening is '[' and closing is ']', return true.
If opening is '{' and closing is '}', return true.
Otherwise, return false.

This version is longer, but very clear.

27. ASCII Version vs Explicit Version

Version	Code style	Beginner friendliness
ASCII trick	Shorter	Slightly harder to understand at first
Explicit matching	Longer	Easier to understand

ASCII version:

```
int diff = closing - opening;  
return diff == 1 || diff == 2;
```

Explicit version:

```
return (opening == '(' && closing == ')') ||  
       (opening == '[' && closing == ']') ||  
       (opening == '{' && closing == '}');
```

Both can work.

For learning, the explicit version is often easier.

For compact code, the ASCII version is shorter.

28. Important Step-by-Step Logic for a Closing Bracket

When the current character is a closing bracket, the algorithm must do these steps in order:

1. Check whether the stack is empty.
2. If empty, return false.
3. If not empty, get the top opening bracket.
4. Pop the stack.
5. Compare the popped opening bracket with the current closing bracket.
6. If they do not match, return false.

Do not pop before checking empty.

That would be unsafe.

29. Why We Store Opening Brackets, Not Closing Brackets

We push opening brackets because they are waiting to be closed later.

Example:

```
{([A+B])}
```

When we see:

```
{
```

it means:

I need a } later.

When we see:

(

it means:

I need a) later.

When we see:

[

it means:

I need a] later.

So opening brackets are stored in the stack until their closing brackets appear.

30. How the Stack Looks During Matching

Expression:

{([A+B])}

Step-by-step stack behavior:

Read {	-> push {	stack: {
Read (	-> push (	stack: { (
Read [	-> push [	stack: { ([
Read]	-> pop [	stack: { (
Read)	-> pop (	stack: {
Read }	-> pop {	stack: empty

The stack becomes empty at the end.

So the expression is balanced.

31. Complete Beginner-Friendly C++ Version Without ASCII Trick

This version may be the clearest for beginners.

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

bool isOpening(char ch) {
    return ch == '(' || ch == '[' || ch == '{';
}

bool isClosing(char ch) {
    return ch == ')' || ch == ']' || ch == '}';
}

bool isMatching(char opening, char closing) {
    return (opening == '(' && closing == ')') ||
           (opening == '[' && closing == ']') ||
           (opening == '{' && closing == '}');
}

bool isBalanced(string exp) {
    stack<char> st;

    for (char ch : exp) {
        if (isOpening(ch)) {
            st.push(ch);
        }
        else if (isClosing(ch)) {
            if (st.empty()) {
                return false;
            }

            char opening = st.top();
            st.pop();

            if (!isMatching(opening, ch)) {
                return false;
            }
        }
    }

    return st.empty();
}
```

```

        }
    }
}

return st.empty();
}

int main() {
    string exp = "{([A+B]*[C-D])}";

    if (isBalanced(exp)) {
        cout << "Balanced" << endl;
    } else {
        cout << "Not Balanced" << endl;
    }

    return 0;
}

```

Expected output:

Balanced

32. Test Examples

Example 1

{([A+B]*[C-D])}

Result:

Balanced

Reason:

Every opening bracket has the correct closing bracket.

Example 2

([{ }])

Result:

Balanced

Reason:

The brackets are nested correctly.

Example 3

([])

Result:

Not Balanced

Reason:

The order is wrong.

[is opened after (, so [must close before (.

Example 4

((A+B))

Result:

Not Balanced

Reason:

There is an extra opening parenthesis.

Example 5

(A+B))

Result:

Not Balanced

Reason:

There is an extra closing parenthesis.

Example 6

{[A+B]}

Result:

Balanced

Reason:

[matches], and { matches }.

Example 7

{[A+B]}

Result:

Not Balanced

Reason:

[is opened but) appears instead of].

33. Time Complexity

Let:

n = length of the expression

The algorithm scans each character once.

For each character, it does a constant amount of work:

- check whether it is opening
- check whether it is closing
- push
- pop
- compare brackets

So the time complexity is:

$O(n)$

This means the running time grows in direct proportion to the number of characters in the expression.

34. Space Complexity

The stack stores opening brackets.

In the worst case, the expression may contain only opening brackets:

((((([[[{ { {

Then the stack may store many characters.

If the expression length is n , the stack may store up to n opening brackets.

So the worst-case space complexity is:

$O(n)$

35. Best Case Space Situation

If the expression has no brackets, such as:

$A+B*C-D$

Then the stack stays empty.

But in Big-O analysis, we usually discuss the worst case.

So the official space complexity is:

$O(n)$

36. Important Beginner Mistakes

Mistake 1: Checking only bracket count

Wrong idea:

If opening count equals closing count, it is balanced.

Why wrong?

Because this expression has equal counts but is not balanced:

$([])$

Mistake 2: Popping without checking empty

Wrong idea:

```
char opening = st.top();  
st.pop();
```

without checking:

```
st.empty()
```

Why wrong?

If the stack is empty, there is no top element.

This can cause an error.

Mistake 3: Forgetting the final stack check

Wrong idea:

```
If no mismatch happens during scanning, return true.
```

Why wrong?

This expression has no mismatch while scanning, but it is still not balanced:

```
((A+B)
```

At the end, one (remains in the stack.

Mistake 4: Ignoring bracket type

Wrong idea:

```
Any opening bracket can match any closing bracket.
```

Why wrong?

[must match], not).

Mistake 5: Misusing the ASCII trick

Wrong idea:

Use ASCII difference without checking opening and closing brackets properly.

Why wrong?

The ASCII trick only works when comparing a valid opening bracket with a valid closing bracket.

You still need `isOpening()` and `isClosing()`.

37. What Lesson 12 Adds to Lesson 9, 10, and 11

Before Lesson 12, the rule was simple:

If '(' appears, push it.
If ')' appears, pop.

Now the rule becomes:

If '(', '[', or '{' appears, push it.
If ')', ']', or '}' appears, pop and check whether the type matches.

The main new idea is:

A closing bracket must match the most recent opening bracket.

38. Key Lesson 12 Formula

For ASCII matching:

`closing - opening == 1 or 2`

Valid examples:

```
')' - '(' = 1  
']' - '[' = 2  
'}' - '{' = 2
```

So in C++:

```
int diff = closing - opening;  
return diff == 1 || diff == 2;
```

39. Key Lesson 12 Sentence

The most important sentence in this lesson is:

The most recently opened bracket must be the first one closed, and it must close with the same bracket type.

This sentence combines both ideas:

1. stack order
2. bracket type matching

40. Beginner Checkpoints

You understand Lesson 12 if you can:

- Explain why a stack is useful for bracket matching.
- Identify opening brackets and closing brackets.
- Explain why `(( [ ] ) )` is not balanced.
- Dry-run `{ ( [ A + B ] * [ C - D ] ) }`.
- Dry-run `{ ( [ A + B ] * [ C - D ] ) }`.
- Explain extra opening bracket errors.
- Explain extra closing bracket errors.
- Explain type mismatch errors.
- Write the valid bracket pairs.
- Explain the ASCII trick.
- Write `isOpening()`, `isClosing()`, and `isMatching()`.
- Explain why the final stack check is needed.
- State the time complexity as `O(n)`.
- State the space complexity as `O(n)`.

41. Practice Tasks

Task 1

Dry-run this expression:

([{ }])

Expected result:

Balanced

Task 2

Dry-run this expression:

([])

Expected result:

Not Balanced

Task 3

Dry-run this expression:

{[(A+B)*(C-D)]}

Expected result:

Balanced

Task 4

Dry-run this expression:

```
{[(A+B)]}
```

Expected result:

```
Not Balanced
```

Reason:

```
( was opened but ] tries to close it.
```

Task 5

Write the function without the ASCII trick:

```
bool isMatching(char opening, char closing) {  
    // write your code here  
}
```

Task 6

Write the function using the ASCII trick:

```
bool isMatching(char opening, char closing) {  
    // write your code here  
}
```

42. Quick Revision Table

Concept	Simple meaning
Opening bracket	A bracket that waits to be closed: (, [, {
Closing bracket	A bracket that closes an opening bracket:),], }
Push	Store an opening bracket on the stack
Pop	Remove the most recent opening bracket
Match	Check whether opening and closing types are correct
Extra opening	Stack is not empty at the end
Extra closing	Closing bracket appears when stack is empty
Mismatch	Popped opening bracket does not match current closing bracket
Time complexity	$O(n)$
Space complexity	$O(n)$

43. Final Summary

Lesson 12 extends parenthesis matching from only:

()

to all three bracket types:

()
[]
{ }

The stack idea is the same:

Opening brackets are pushed.
Closing brackets cause a pop.

The new rule is:

After popping, the popped opening bracket must match the current closing bracket.

So:

```
( matches )  
[ matches ]  
{ matches }
```

The algorithm returns balanced only if:

1. no extra closing bracket appears,
2. no mismatch appears,
3. no opening bracket remains at the end.

The final result is:

Use a stack to remember unmatched opening brackets.
Whenever a closing bracket appears, match it with the most recent opening bracket.
At the end, the stack must be empty.

That is the full idea of Lesson 12.